Apple Pay Express Checkout (ECS)

Integration Guide — Braintree Mobile SDK v7+ (iOS)

Includes full API interaction map and Apple Pay address-retrieval reference · June 12, 2026 · Braintree iOS SDK 7.7.0

1. Overview

In the express checkout (ECS) flow, the buyer taps the Apple Pay button directly on a product or cart page. The Apple Pay sheet — not a merchant checkout form — collects the shipping address, contact details and shipping method. The merchant app reacts to sheet events to recalculate shipping and tax, tokenizes the authorized payment through Braintree, and completes the sale server-side. No merchant review page is needed.

Actors and responsibilities:

- **Merchant iOS app —** presents PKPaymentAuthorizationController, implements its delegate, extracts addresses, calls your server.
- **Braintree SDK (BTApplePayClient, v7+) —** builds the PKPaymentRequest from your gateway config and tokenizes the PKPayment into a payment-method nonce.
- **Apple Pay (PassKit + Apple servers) —** renders the sheet, manages address/shipping selection, returns the encrypted payment token.
- **Merchant server —** issues the client token and creates the transaction with the nonce + address.
- **Braintree Gateway —** decrypts the Apple Pay token and processes the payment.

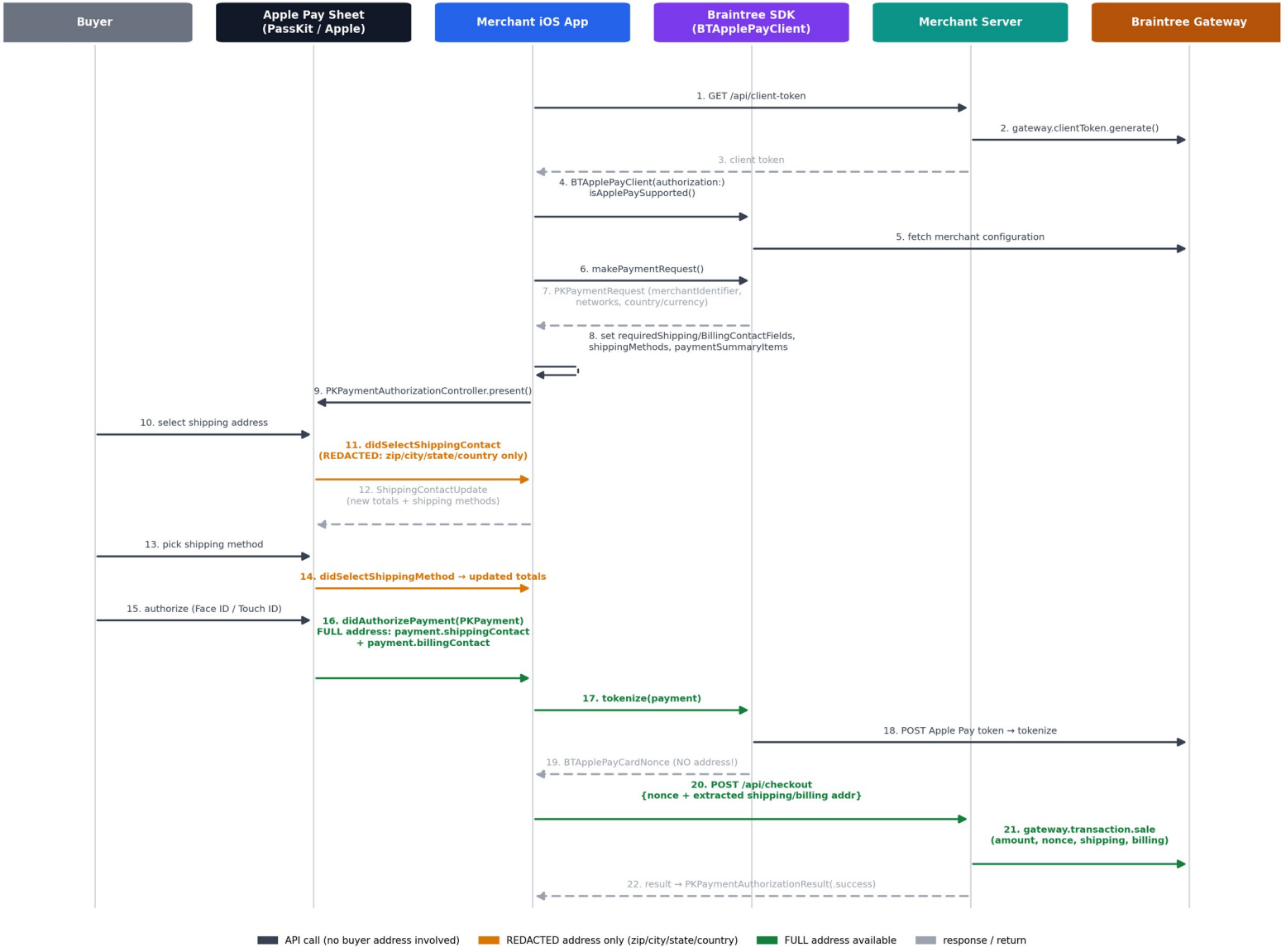
v7 note: clients are constructed directly with a tokenization key or client token — BTAPIClient is no longer passed in (BTApplePayClient(authorization:)). All v7 releases also carry the updated SSL certificates required after March 30, 2026.

2. Prerequisites (one-time setup)

Where	What to do
Apple Developer portal	Create an Apple Merchant ID (e.g. merchant.com.yourstore.app).
Braintree Control Panel	Settings → Processing → Apple Pay → register your Apple Merchant ID (upload the CSR Braintree provides, download the certificate back to the portal).
Xcode project	Signing & Capabilities → add the Apple Pay capability and tick your Merchant ID.
SDK install	Swift Package Manager: https://github.com/braintree/braintree_ios (module BraintreeApplePay), or CocoaPods: pod 'Braintree/ApplePay'. Use v7+.
Sandbox testing	Real device + sandbox iCloud tester account with a test card in Wallet (simulator does not exercise real tokens).

3. API Interaction Map

End-to-end sequence. Color encodes the address state of each call: gray = no buyer address involved, orange = redacted address only (zip / city / state / country), green = full address available. Dashed arrows are responses.



3.1 Step reference

Step	Interaction	API	Purpose	Address state
1–3	App → Server → BT Gateway	GET /api/client-token → gateway.clientToken.generate()	Fetch authorization for the SDK. Alternatively embed a tokenization key.	—
4–5	App → BT SDK → Gateway	BTApplePayClient(authorization:) · isApplePaySupported()	Init client; confirm Apple Pay is enabled for the merchant account AND the device can pay.	—
6–7	App → BT SDK	makePaymentRequest()	Returns a PKPaymentRequest pre-filled with merchantIdentifier, supportedNetworks, country/currency from gateway config.	—
8	App (local)	Set requiredShippingContactFields / requiredBillingContactFields, shippingMethods, paymentSummaryItems	THE step that makes Apple Pay return addresses. No fields requested = no address returned.	—
9	App → PassKit	PKPaymentAuthorizationController.present()	Show the Apple Pay sheet.	—
10–12	Sheet → App delegate	didSelectShippingContact(PKContact)	Buyer picked/changed address. Recalculate shipping + tax; return PKPaymentRequestShippingContactUpdate.	**REDACTED: postalCode, locality, administrativeArea, country only — street and name are withheld pre-authorization.**
13–14	Sheet → App delegate	didSelectShippingMethod(PKShippingMethod)	Buyer picked a shipping method; return updated summary items.	**REDACTED (unchanged)**
15–16	Buyer → Sheet → App delegate	didAuthorizePayment(PKPayment)	Face ID / Touch ID approved. PKPayment carries the encrypted token AND the complete contacts.	**FULL: payment.shippingContact (street, name, email, phone) + payment.billingContact.**
17–19	App → BT SDK → Gateway	tokenize(payment) → BTApplePayCardNonce	Exchange the Apple Pay token for a Braintree nonce. The nonce does NOT carry the shipping address — extract it from PKPayment yourself.	App holds FULL address; nonce has none.
20–21	App → Server → Gateway	POST /api/checkout {nonce, shipping, billing} → gateway.transaction.sale(...)	Create the transaction with the address attached (AVS, fraud tools, shipping record).	**FULL (sent by app)**
22	App delegate → Sheet	PKPaymentAuthorizationResult(.success)	Close the sheet with the success animation (or .failure with errors).	—

4. Client Implementation (Swift, v7 API)

4.1 Initialize and build the payment request (steps 1–9)

```
import BraintreeApplePay
import PassKit

let applePayClient = BTApplePayClient(authorization: clientToken) // v7: no BTAPIClient

func startApplePayECS() async throws {
    guard await applePayClient.isApplePaySupported() else { return } // hide the button

    var request = try await applePayClient.makePaymentRequest()
    // pre-filled: merchantIdentifier, supportedNetworks, countryCode, currencyCode

    // *** ECS: ask the sheet to collect the address — without this you get nothing ***
    request.requiredShippingContactFields = [.postalAddress, .name, .emailAddress, .phoneNumber]
    request.requiredBillingContactFields  = [.postalAddress]

    request.shippingMethods = [
        shippingMethod("Standard (3-5 days)", "0.00",  id: "standard"),
        shippingMethod("Express (2 days)",    "12.99", id: "express")
    ]
    request.paymentSummaryItems = summaryItems(shipping: request.shippingMethods![0])

    let controller = PKPaymentAuthorizationController(paymentRequest: request)
    controller.delegate = self
    controller.present()
}
```

4.2 React to address / shipping selection (steps 10–14)

Pre-authorization callbacks receive a REDACTED address — enough for shipping cost and tax, nothing more. Do not expect a street address here.

```
func paymentAuthorizationController(_ controller: PKPaymentAuthorizationController,
        didSelectShippingContact contact: PKContact) async
        -> PKPaymentRequestShippingContactUpdate {
    // REDACTED: only postalCode / locality / administrativeArea / country are populated
    let zip   = contact.postalAddress?.postalCode ?? ""
    let state = contact.postalAddress?.state ?? ""
    let (methods, items) = recalcShippingAndTax(zip: zip, state: state)
    return PKPaymentRequestShippingContactUpdate(
        errors: nil, paymentSummaryItems: items, shippingMethods: methods)
}

func paymentAuthorizationController(_ controller: PKPaymentAuthorizationController,
        didSelectShippingMethod shippingMethod: PKShippingMethod) async
        -> PKPaymentRequestShippingMethodUpdate {
    PKPaymentRequestShippingMethodUpdate(
        paymentSummaryItems: summaryItems(shipping: shippingMethod))
}
```

4.3 Authorization: get the FULL address + tokenize (steps 15–19)

The complete address exists ONLY on the PKPayment inside didAuthorizePayment. BTApplePayCardNonce does not copy it — extract it here and ship it to your server with the nonce.

```
func paymentAuthorizationController(_ controller: PKPaymentAuthorizationController,
```

```
        didAuthorizePayment payment: PKPayment) async -> PKPaymentAuthorizationResult {
    do {
        let nonce = try await applePayClient.tokenize(payment)   // BTApplePayCardNonce

        // FULL shipping address — now includes street + name + email + phone
        let ship = payment.shippingContact
        let a    = ship?.postalAddress
        let shippingAddress: [String: String?] = [
            "recipientName": (ship?.name).map {
                PersonNameComponentsFormatter().string(from: $0) },
            "email":   ship?.emailAddress,
            "phone":   ship?.phoneNumber?.stringValue,
            "street":  a?.street,            // multi-line: split on "\n" if needed
            "city":    a?.city,
            "state":   a?.state,
            "zip":     a?.postalCode,
            "country": a?.isoCountryCode
        ]
        let billingZip = payment.billingContact?.postalAddress?.postalCode  // for AVS

        try await api.checkout(nonce: nonce.nonce,
                               shipping: shippingAddress,
                               billingZip: billingZip,
                               shippingMethodId: payment.shippingMethod?.identifier)
        return PKPaymentAuthorizationResult(status: .success, errors: nil)
    } catch {
        return PKPaymentAuthorizationResult(status: .failure, errors: [error])
    }
}

func paymentAuthorizationControllerDidFinish(_ controller: PKPaymentAuthorizationController) {
    controller.dismiss()
}
```

5. Getting the Address from Apple Pay — the Four Rules

Rule 1 — Request it or lose it. Addresses are returned only for fields named in requiredShippingContactFields / requiredBillingContactFields (step 8). An empty set means the sheet never asks the buyer for an address.

Rule 2 — Before authorization the address is redacted. didSelectShippingContact receives only zip, city, state and country — Apple withholds street and name until the buyer authorizes. Use it solely for shipping/tax math; never validate deliverability against it.

Rule 3 — The full address lives on PKPayment, not on the nonce. After Face ID, payment.shippingContact and payment.billingContact carry the complete contacts. BTApplePayClient.tokenize() returns only a payment nonce — if you don't extract the address in didAuthorizePayment, it is gone.

Rule 4 — Forward it to Braintree. Pass shipping + billing into transaction.sale so the gateway stores the shipping record and runs AVS / fraud checks (billing zip at minimum).

6. Server Side (steps 20–21)

Identical to any other Braintree nonce — platform-agnostic. Node example:

```
app.post("/api/checkout", async (req, res) => {
  const { nonce, amount, shipping, billingZip } = req.body;
```

```
  const result = await gateway.transaction.sale({
    amount,
    paymentMethodNonce: nonce,
    shipping: {
      firstName: shipping.recipientName?.split(" ")[0],
      lastName:  shipping.recipientName?.split(" ").slice(1).join(" "),
      streetAddress: shipping.street,
      locality: shipping.city,
      region: shipping.state,
      postalCode: shipping.zip,
      countryCodeAlpha2: shipping.country
    },
    billing: { postalCode: billingZip },          // AVS
    options: { submitForSettlement: true }
  });
  res.json({ ok: result.success, id: result.transaction?.id });
});
```

Re-verify the amount server-side (recompute totals from the cart + shipping method id); never trust a client-supplied total.

7. Testing & Go-Live Checklist

- Sandbox: real device signed into a sandbox iCloud tester account with an Apple Pay test card in Wallet.
- Verify isApplePaySupported() returns true (Apple Pay enabled on the Braintree merchant account; correct merchant account currency).
- Confirm didSelectShippingContact totals update for different zips (tax) and that errors (e.g. unsupported country) surface via PKPaymentRequestShippingContactUpdate(errors:).
- Confirm the full street address arrives in didAuthorizePayment and lands on the Braintree transaction (Control Panel → transaction → shipping details).
- Production: switch the authorization (tokenization key / client token) to production credentials and re-register the production Apple Merchant ID certificate.

8. References

- BTApplePayClient (7.7.0) — https://braintree.github.io/braintree_ios/current/Classes/BTApplePayClient.html
- Braintree iOS SDK — https://github.com/braintree/braintree_ios
- Braintree Apple Pay guide (client-side, iOS) — https://developer.paypal.com/braintree/docs/guides/apple-pay/client-side/ios/v5/
- Apple: PKPaymentAuthorizationController / PKContact — https://developer.apple.com/documentation/passkit